# 2

# The Transformer: The Model Behind the Modern AI Revolution

In this chapter, we will discuss the limitations of the models we saw in the previous chapter, and how a new paradigm (first attention mechanisms and then the transformer) emerged to solve these limitations. This will enable us to understand how these models are trained and why they are so powerful. We will discuss why this paradigm has been successful and why it has made it possible to solve tasks in **natural language processing** (**NLP**) that were previously impossible. We will then see the capabilities of these models in practical application.

This chapter will clarify why contemporary LLMs are inherently based on the transformer architecture.

In this chapter, we'll be covering the following topics:

- Exploring attention and self-attention
- Introducing the transformer model
- Training a transformer
- Exploring masked language modeling
- Visualizing internal mechanisms
- Applying a transformer

## Technical requirements

Most of this code can be run on a CPU, but some parts (fine-tuning and knowledge distillation) are preferable to be run on a GPU (one hour of

training on a CPU versus less than five minutes on a GPU).

The code is written in PyTorch and uses standard libraries for the most part (PyTorch, Hugging Face Transformers, and so on), though some snippets come from Ecco, a specific library. The code can be found on GitHub: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr2**

# Exploring attention and self-attention

In the 1950s, with the beginning of the computer revolution, governments began to become interested in the idea of machine translation, especially for military applications. These attempts failed miserably, for three main reasons: machine translation is more complex than it seems, there was not enough computational power, and there was not enough data. Governments concluded that it was a technically impossible challenge in the 1960s.

By the 1990s, two of the three limitations were beginning to be overcome: the internet finally allowed for abundant text, and the advent of GPUs finally allowed for computational power. The third requirement still had to be met: a model that could harness the newfound computational power to handle the complexity of natural language.

Machine translation captured the interest of researchers because it is a practical problem for which it is easy to evaluate the result (we can easily understand whether a translation is good or not). Moreover, we have an abundance of text in one language and a counterpart in another. So, researchers tried to adapt the previous models to the tasks (RNN, LSTM, and so on). The most commonly used system was the **seq2seq model**, where you have an **encoder** and a **decoder**. The encoder transforms the sequence into a new succinct representation that should preserve the relevant information (a sort of good summary). The decoder receives as input this context vector and uses this to transform (translate) this input into the output.

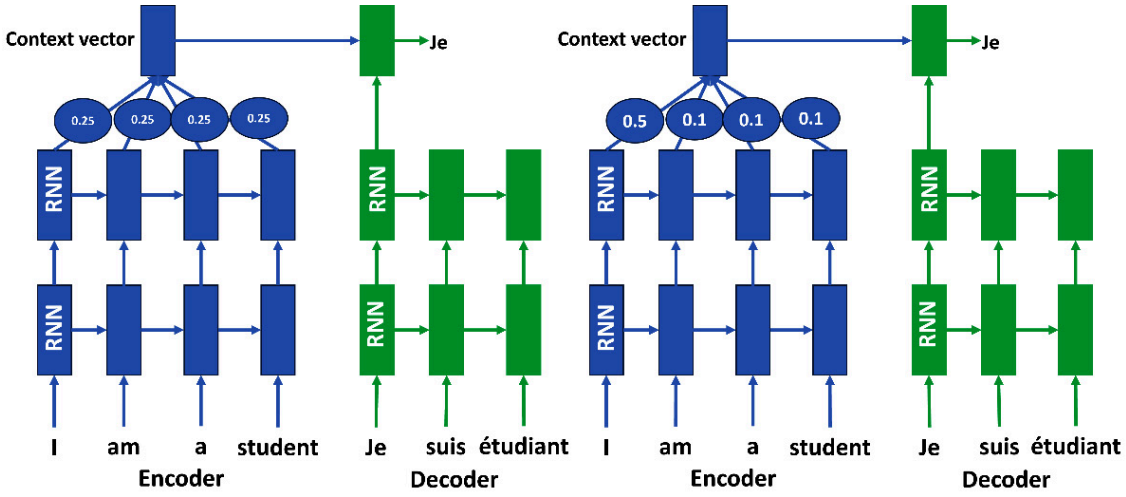



Figure 2.1 – A seq2seq model with an encoder and a decoder. In one case, we take the average of the hidden states (left); in the other case, we use attention to identify which hidden state is more relevant for the translation (right)

RNNs and derived models have some problems:

- **Alignment**: The length of input and output can be different (for example, to translate English to French "*she doesn't like potatoes*" into "*elle n'aime pas les pommes de terre*").
- **Vanishing and exploding gradients**: Problems that arise during training so that multiple layers cannot be managed effectively.
- **Non-parallelizability**: Training is computationally expensive and not parallelizable. RNNs forget after a few steps.

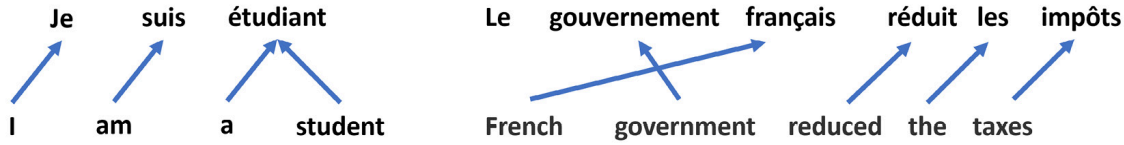



Figure 2.2 – Example of issues with alignment: one to many (left) and spurious word (right)

**Attention mechanisms** were initially described to solve the alignment problem, as well as to learn the relationships between the various parts of a text and the corresponding parts of the translated text.

The idea is that instead of passing the hidden state of RNNs, we pass contextual information that focuses only on the important parts of the sequence. During decoding (translation) for each token, we want to retrieve the corresponding and specific information in the other language.

Attention determines which tokens in the input are important at that moment.

The first step is the alignment between the hidden state of the encoder (*h*) and the previous decoder output (*s*). The score function can be different: dot product or cosine similarity is most commonly used, but it can also be more complex functions such as the feedforward neural network layer. This step allows us to understand how relevant hidden state encoders are to the translation at that time. This step is conducted for all encoder steps.

$$1) e_{i,j} = score(s_{i-1}, h_j)$$

Right now though, we have a scalar representing the similarity between two vectors (*h* and *s*). All these scores are passed into the softmax function that squeezes everything between 0 and 1. This step also serves to assign relative importance to each hidden state.

$$2) a_{i,j} = softmax\, e_{i,j}) = \frac{\exp(e_{i,j})}{\sum_{k=1}^{t} \exp(e_{i,k})}$$

Finally, we conduct a weighted sum of the various hidden states multiplied by the attention score. So, we have a fixed-length context vector capable of giving us information about the entire set of hidden states. In simple words, during translation, we have a context vector that is dynamically updated and tells us how much attention we should give to each part of the input sequence.

$$3) c_t = \sum_{j=1}^{T} a_{i,j} \bullet h_j$$

As you can see from the original article, the model pays different attention to the various words in the input during translation.

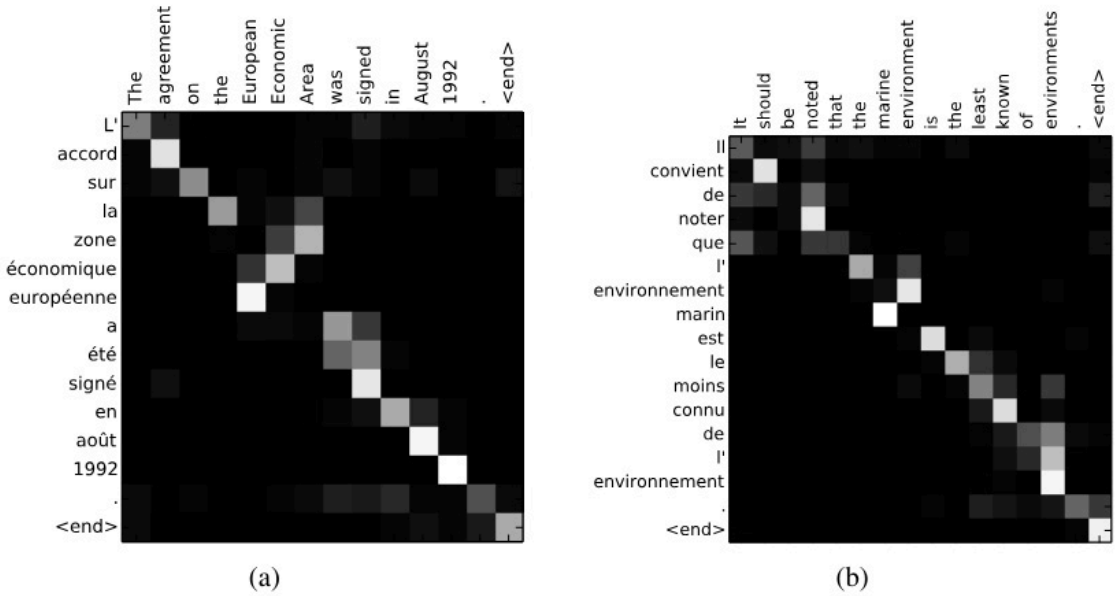



Figure 2.3 – Example of alignment between sentences after training the model with attention. Each pixel shows the attention weight between the source word and the target word. (**https://arxiv.org/pdf/1409.0473**)

In addition to solving alignment, the attention mechanism has other advantages:

- It reduces the vanishing gradient problem because it provides a short-cut to early states.
- It eliminates the bottleneck problem; the encoder can directly go to the source in the translation.
- It also provides interpretability because we know which words are used for alignment.
- It definitely improves the performance of the model.

Its success has given rise to several variants where the scoring function is different. One variant in particular, called **self-attention**, has the particular advantage that it extracts information directly from the input without necessarily needing to compare it with something else.

The insight behind self-attention is that if we want to look for a book for an essay on the French Revolution in a library (query), we don't need to read all the books to find a book on the history of France (value), we just need to read the coasts of the books (key). Self-attention, in other words, is a method that allows us to search within context to find the representation we need.

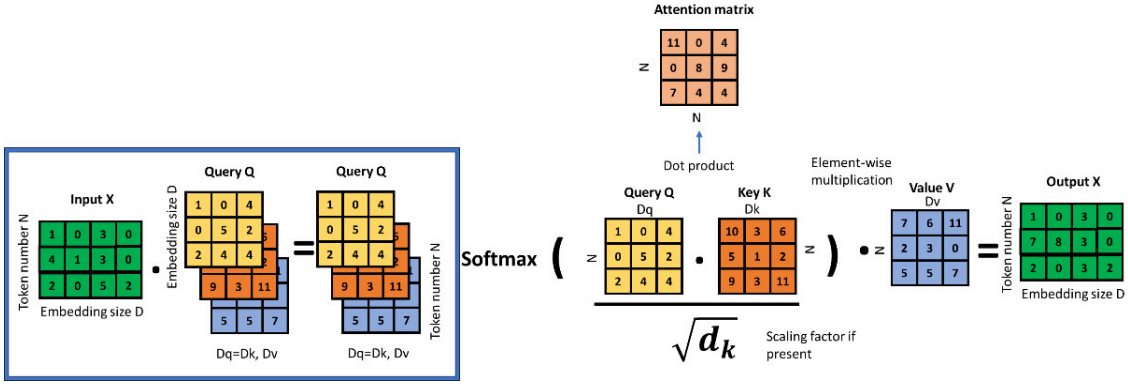



Figure 2.4 – Self-attention mechanism. Matrix dimensions are included; numbers are arbitrary

Transacting this for a model, given an input we want to conduct a series of comparisons between the various components of the sequence (such as tokens) to obtain an output sequence (which we can then use for various models or tasks). The self-attention equation is as follows:

$$Attention\,Q, K, V) = softmax\,\frac{Q \bullet k^T}{\sqrt{d_k}}\Big) \bullet V$$

You can immediately see that it is derived from the original attention formula. We have the dot product to conduct comparisons, and we then exploit the **`softmax`** function to calculate the relative importance and normalize the values between 0 and 1. *D* is the size of the sequence; in other words, self-attention is also normalized as a function of the length of our sequence.

The next step is using **`softmax`**. Here's a little refresher on the function (how you calculate and how it is implemented more efficiently in Python):

$$y = \frac{e^{x_i}}{\sum_{i=1}^{i=n} e^{x_i}}$$

$$y = \begin{matrix} y_1 \\ y_2 \\ y_3 \end{matrix}\Big] = softmax\,x) = f\begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix}\Big]\Big) = \begin{matrix} \frac{e^{x_1}}{e^{x_1}+e^{x_2}+e^{x_3}} \\ \frac{e^{x_2}}{e^{x_1}+e^{x_2}+e^{x_3}} \\ \frac{e^{x_3}}{e^{x_1}+e^{x_2}+e^{x_3}} \end{matrix}\Big]$$

$$python: y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = softmax\,x) = f\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \frac{x_1 \quad x_2 \quad x_3]}{e^{x_1} + e^{x_2} + e^{x_3}} = \frac{e^x}{sum(e^x)}$$

As we saw in the previous chapter, the dot product can become quite wide as the length of the vectors increases. This can lead to inputs that are too large in the **`softmax`** function (this shifts the probability mass in **`softmax`** to a few elements and thus leads to small gradients). In the original article, they solved this by normalizing by the square root of *D*.

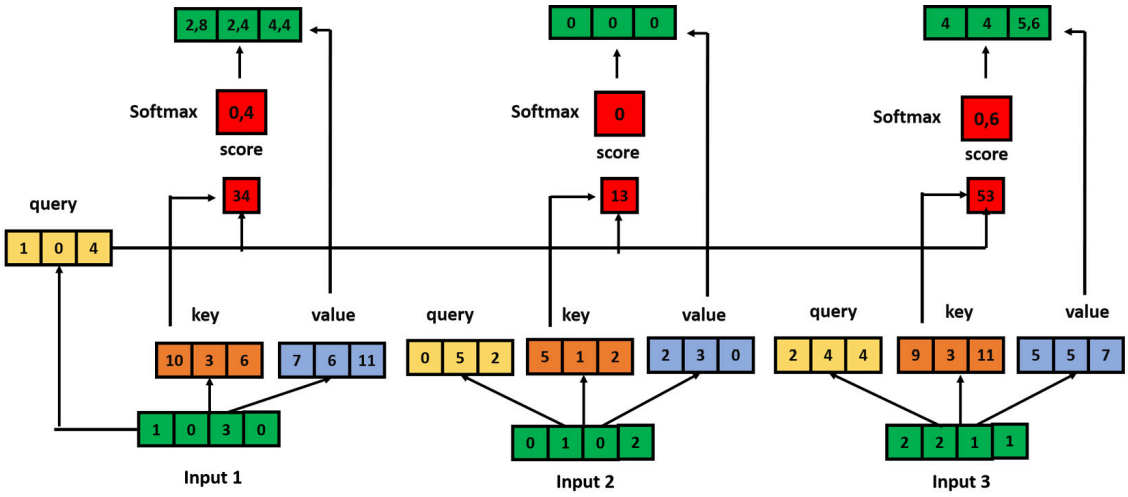



Figure 2.5 – Self-attention unrolled

The real difference is that we use three matrices of weights **Query** (**Q**), **Key** (**K**), and **Value** (**V**) that are initially randomly initialized. *Q* is the current focus of attention, while *K* informs the model about previous inputs, and *V* serves to extract the final input information. So, the first step is the multiplication of these three matrices with our input *X* (an array of vectors, of which each represents a token).

$$\boldsymbol{Q = X \bullet W^Q, K = X \bullet W^V, V = X \bullet W^V}$$

The beauty of this system is that we can use it to extract more than one representation from the same input (after all, we can have multiple questions in a textbook). Therefore, since the operations are parallelizable, we can have multi-head attention. **Multi-head self-attention** enables the model to simultaneously capture multiple types of relationships within the input sequence. This is crucial because a single word in a sentence can be contextually related to several other words. During training, the *K* and *Q* matrices in each head specialize in modeling different kinds of re-

lationships. Each attention head produces an output based on its specific perspective, resulting in *n* outputs for *n* heads. These outputs are then concatenated and passed through a final linear projection layer to restore the dimensionality backt to the original input size.

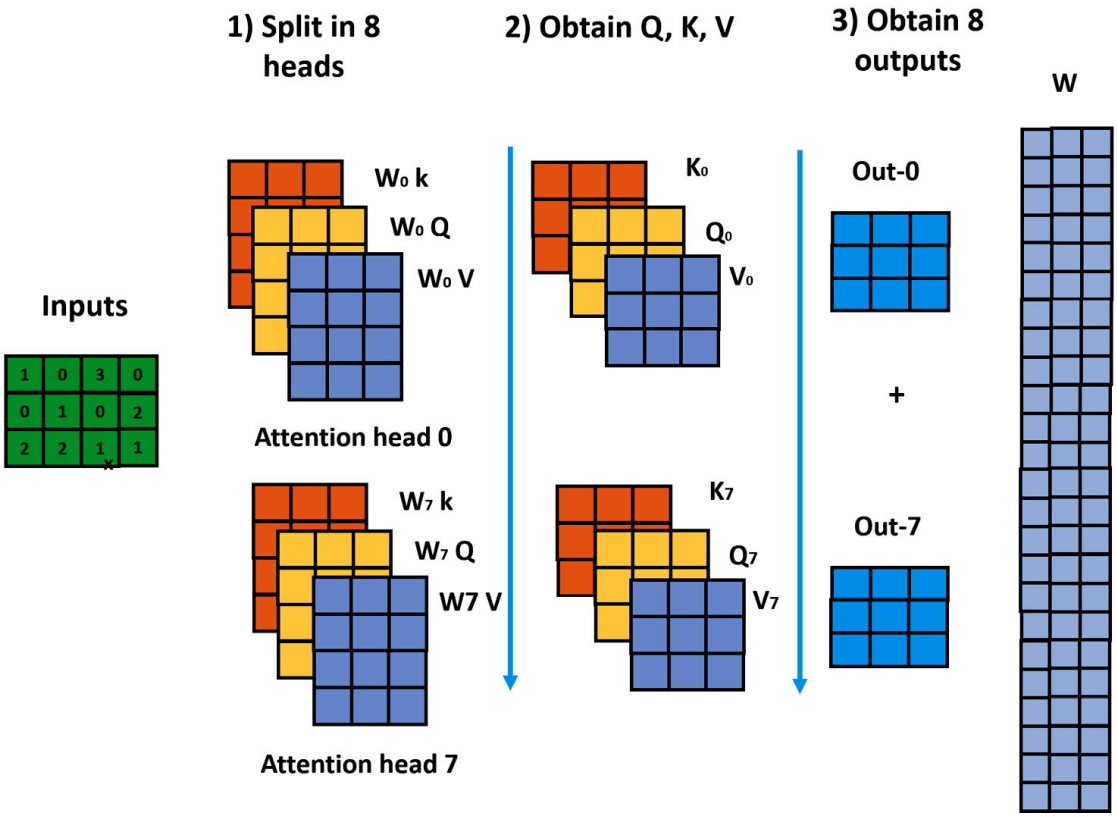



Figure 2.6 – Multi-head self-attention

Self-attention has several advantages:

- We can extract different representations for each input.
- We can conduct all these computations in parallel and thus with a GPU. Each head can be computed independently.
- We can use it in models that do not necessarily consist of an encoder and a decoder.
- We do not have to wait for different time steps to see the relationship between distant word pairs (as in RNN).
- However, it has a quadratic cost in function of the number of tokens *N*, and it has no inherent notion of order.

Self-attention is computationally expensive. It can be shown that, considering a sequence *T* and sequence length *d*, the computation cost and space is quadratic:

$$time = \mathcal{O}\left(T^2 + d\right) space = \mathcal{O}\left(T^2 + Td\right)$$

They identified the dot product as the culprit. This computational cost is one of the problems of scalability (taking into account that multi-head attention is calculated in each block). For this reason, many variations of self-attention have been proposed to reduce the computational cost.

Despite the computational cost, self-attention has shown its capability, especially when several layers are stacked on top of each other. In the next section, we will discuss how this makes the model extremely powerful despite its computational cost.

# Introducing the transformer model

Despite this decisive advance though, several problems remain in machine translation:

- The model fails to capture the meaning of the sentence and is still error-prone
- In addition, we have problems with words that are not in the initial vocabulary
- Errors in pronouns and other grammatical forms
- The model fails to maintain context for long texts
- It is not adaptable if the domain in the training set and test data is different (for example, if it is trained on literary texts and the test set is finance texts)
- RNNs are not parallelizable, and you have to compute sequentially

Considering these points, Google researchers in 2016 came up with the idea of eliminating RNNs altogether rather than improving them. According to the authors of the *Attention is All You Need* seminal article; all you need is a model that is based on multi-head self-attention. Before going into detail, the transformer consists entirely of stacked layers of multi-head self-attention. In this way, the model learns a hierarchical and increasingly sophisticated representation of the text.

The first step in the process is the transformation of text into numerical vectors (tokenization). After that, we have an embedding step to obtain vectors for each token. A special feature of the transformer is the introduction of a function to record the position of each token in the sequence (self-attention is not position-aware). This process is called **positional encoding**. The authors in the article use sin and cos alternately with position. This allows the model to know the relative position of each token.

$$PE_{pos,2i} = \sin(pos/10002i/d)$$

$$PE_{pos,2i+1} = \cos(pos/10002i/d)$$

In the first step, the embedding vectors are summed with the result of these functions. This is because self-attention is not aware of word order, but word order in a period is important. Thus, the order is directly encoded in the vectors it awaits. Note, though, that there are no learnable parameters in this function and that for long sequences, it will have to be modified (we will discuss this in the next chapter).

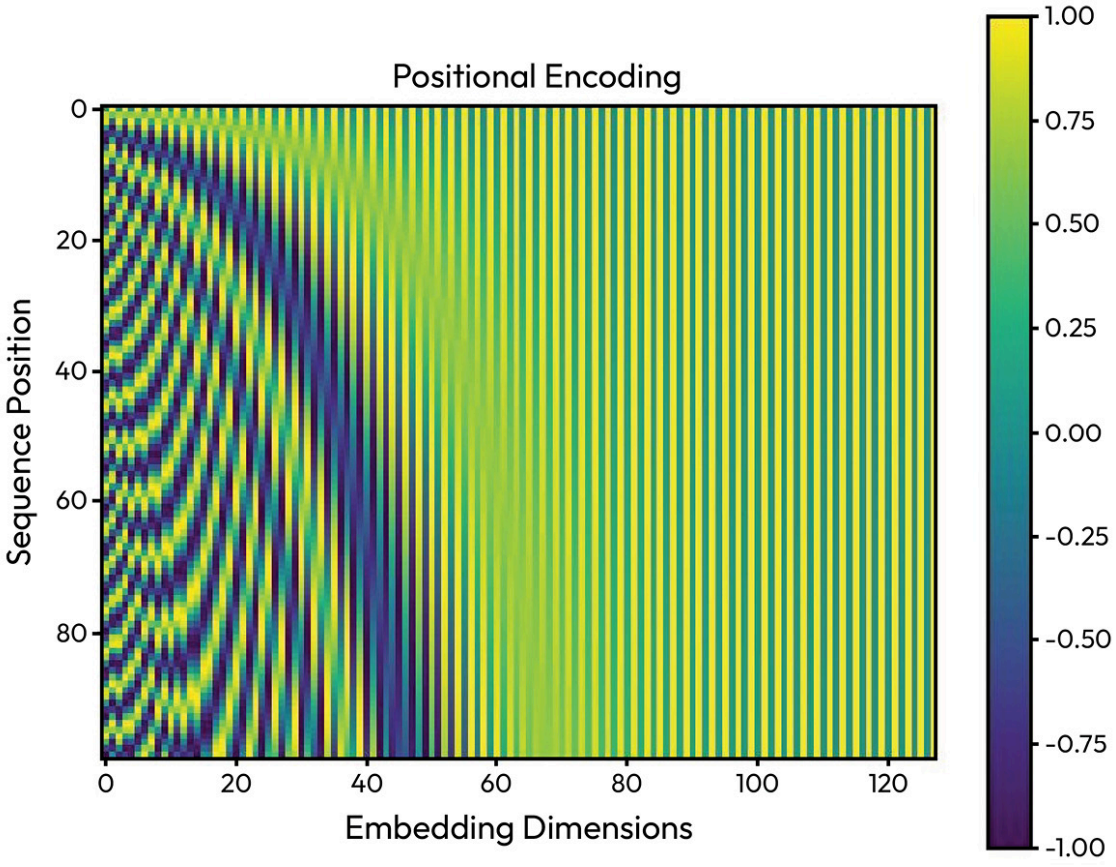



Figure 2.7 – Positional encoding

After that, we have a series of transformer blocks in sequence. The **transformer block** consists of four elements: multi-head self-attention, feed-forward layer, residual connections, and layer normalization.

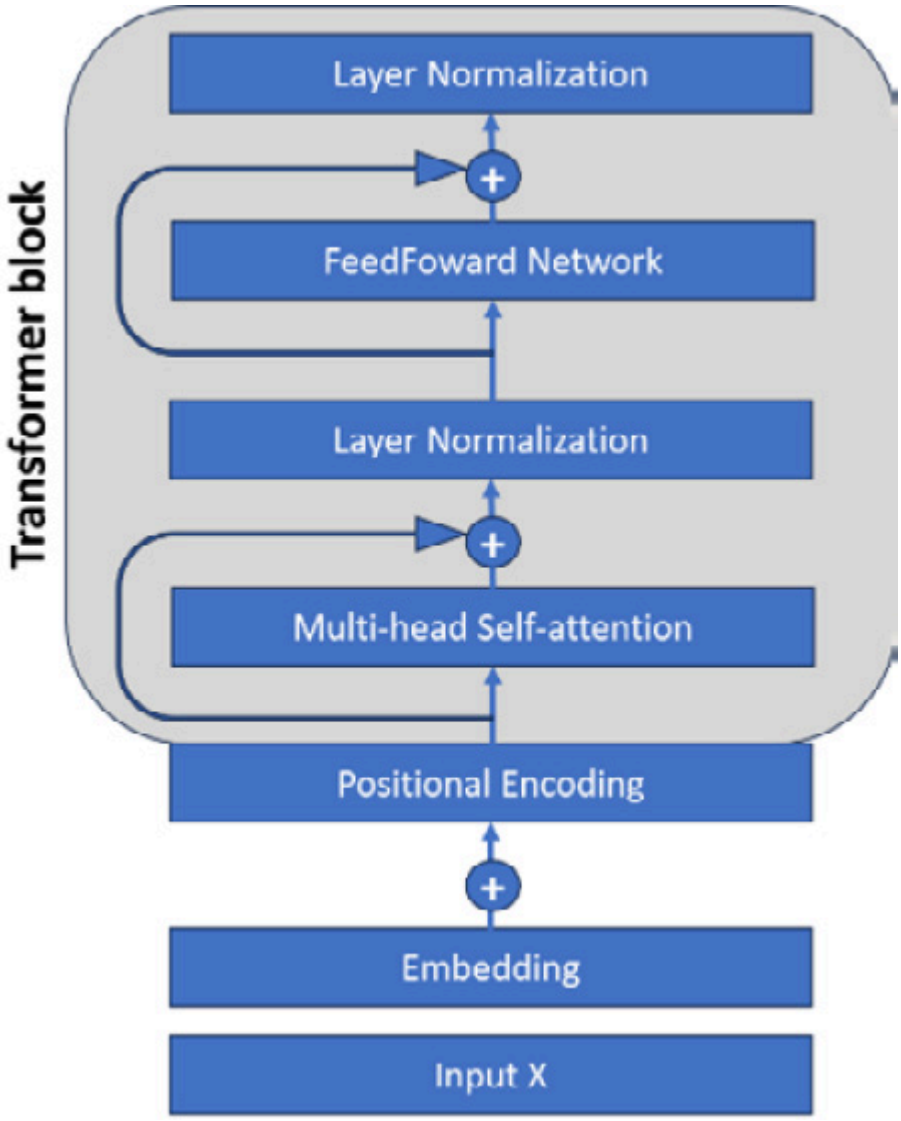



Figure 2.8 – Flow diagram of the transformer block

The feedforward layer consists of two linear layers. This layer is used to obtain a linear projection of the multi-head self-attention. The weights are identifiable for each position and are separated. It can be seen as two linear transformations with one ReLU activation in between.

$$FFNx=max(0,xW1+b1)W2+b2$$

This adds a step of non-linearity to self-attention. The FFN layer is chosen because it is an easily parallelized operation.

Residual connections are connections that pass information between two layers without going through the intermediate layer transformation.

Initially developed in convolutional networks, they allow a shortcut between layers and help the gradient pass down to the lower layers. In the transformer, blocks are present for both the attention layer and feedforward, where the input is summed with the output. Residual connections also have the advantage of making the loss surface smoother (this helps the model find a better minimum and not get stuck in a local loss). This powerful effect can be seen clearly in *Figure 2.9*:

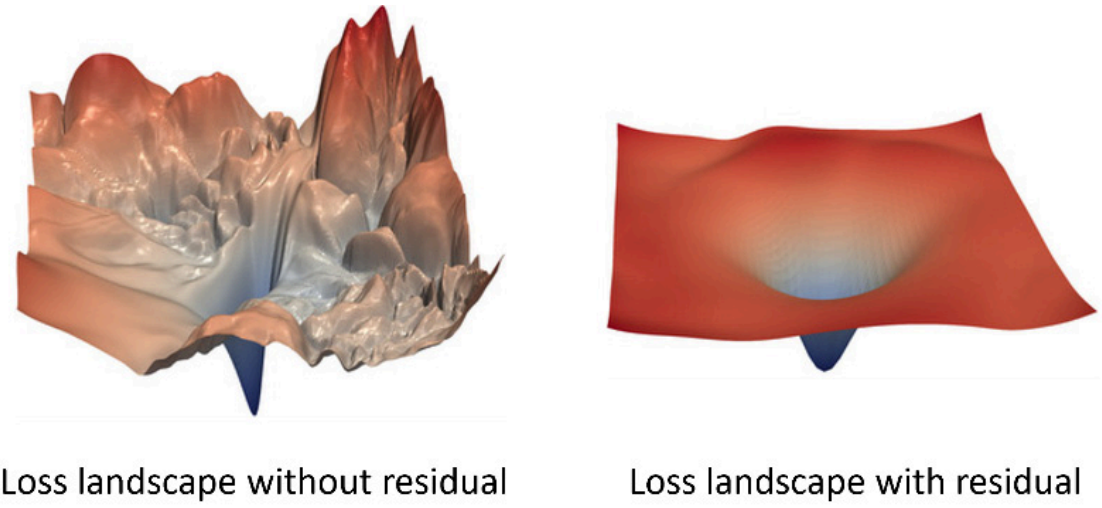



Figure 2.9 – Effect of the residual connections on the loss

*NOTE*

*Figure 2.9 is originally from Visualizing the Loss Landscape of Neural Nets by Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein (https://github.com/tomgoldstein/loss-landscape/tree/master).*

The residual connection makes the loss surface smoother, which allows the model to be trained more efficiently and quickly.

Layer normalization is a form of normalization that helps training because it keeps the hidden layer values in a certain range (it is an alternative to batch normalization). Having taken a single vector, it is normalized in a process that takes advantage of the mean and standard deviation. Having calculated the mean and standard deviation, the vector is scaled:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\widehat{x} = \frac{(x - \mu)}{\sigma}$$

In the final transformation, we exploit two parameters that are learned during training.

$$LayerNormalization = \gamma\widehat{x} + \beta$$

There is a lot of variability during the training, and this can hurt the learning of the training. To reduce uninformative variability, we add this normalization step, thus normalizing the gradient as well.

At this point, we can assemble everything into a single block. Consider that after embedding, we have as input *X* a matrix of dimension *n x d* (with *n* being the number of tokens, and *d* the dimensions of the embedding). This input *X* goes into a transformer block and comes out with the same dimensions. This process is repeated for all transformer blocks:

H=LayerNormX+MultiHeadSelfAttentionX

H=LayerNormH+FFNH

Some notes on this process are as follows:

- In some architectures, *LayerNorm* can be after the *FFN* block instead of before (whether it is better or not is still debated).
- Modern models have up to 96 transformer blocks in series, but the structure is virtually identical. The idea is that the model learns an increasingly complex representation of the language.
- Starting with the embedding of an input, self-attention allows this representation to be enriched by incorporating an increasingly complex context. In addition, the model also has information about the location of each token.
- Absolute positional encoding has the defect of overrepresenting words at the beginning of the sequence. Today, there are variants that consider the relative position.

Once we have “the bricks,” we can assemble them into a functional structure. In the original description, the model was structured for machine translation and composed of two parts: an encoder (which takes the text to be translated) and a decoder (which will produce the translation).

The original transformer is composed of different blocks of transformer blocks and structures in an encoder and decoder, as you can see in *Figure 2.10*.

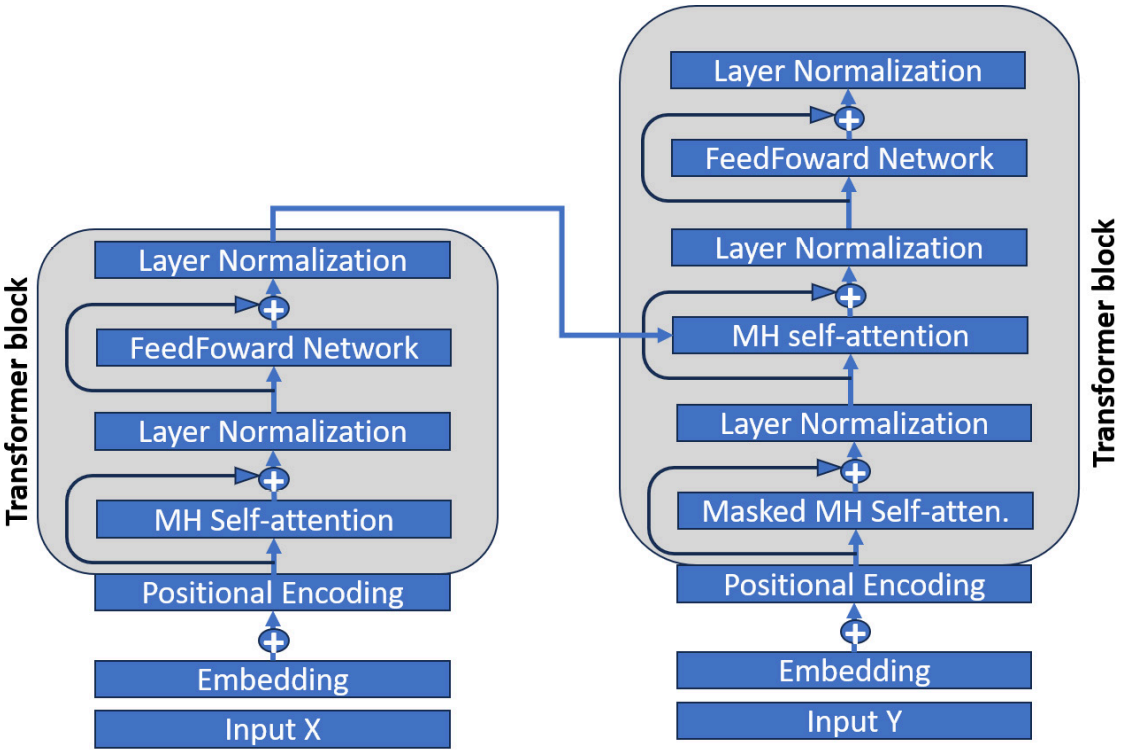



Figure 2.10 – Encoder-decoder structure

The decoder, like the encoder, is composed of an embedding, a positional encoder, and a series of transformer blocks. One note is that in the decoder, instead of self-attention, we have **cross-attention**. Cross-attention is exactly the same, only we take both elements from the encoder and the decoder (because we want to condition the generation of the decoder based on the encoder input). In this case, the queries come from the encoder and the rest from the decoder. As you can see from *Figure 2.11*, the decoder sequence can be of different sizes, but the result is the same:

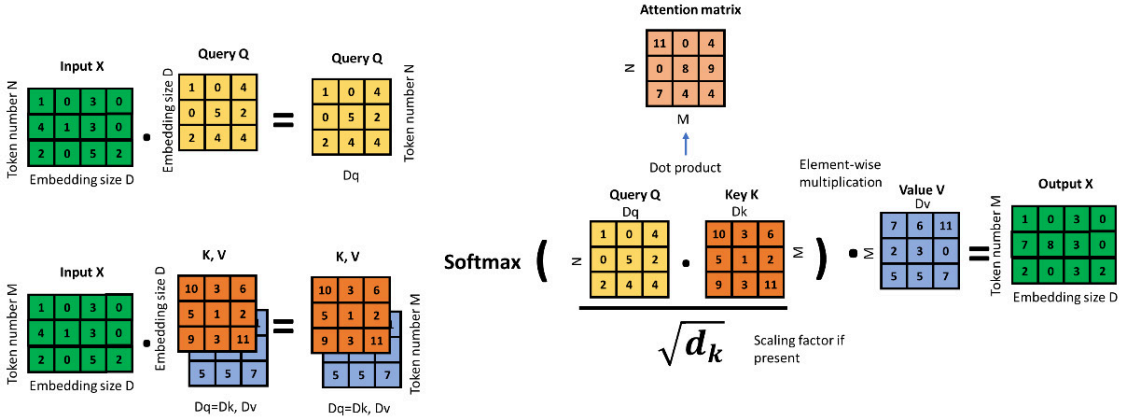



Figure 2.11 – Cross-attention

Input *N* comes from the encoder, while input *M* is from the decoder. In the figure, cross-attention is mixing information from the encoder and decoder, allowing the decoder to learn from the encoder.

Another note on the structure: in the decoder, the first self-attention has an additional mask to prevent the model from seeing the future.

This is especially true in the case of QT. In fact, if one wants to predict the next word and the model already knows what it is, we have data leakage. To compensate for this, we add a mask in which the upper-triangular portion is replaced with negative infinity: - ∞.

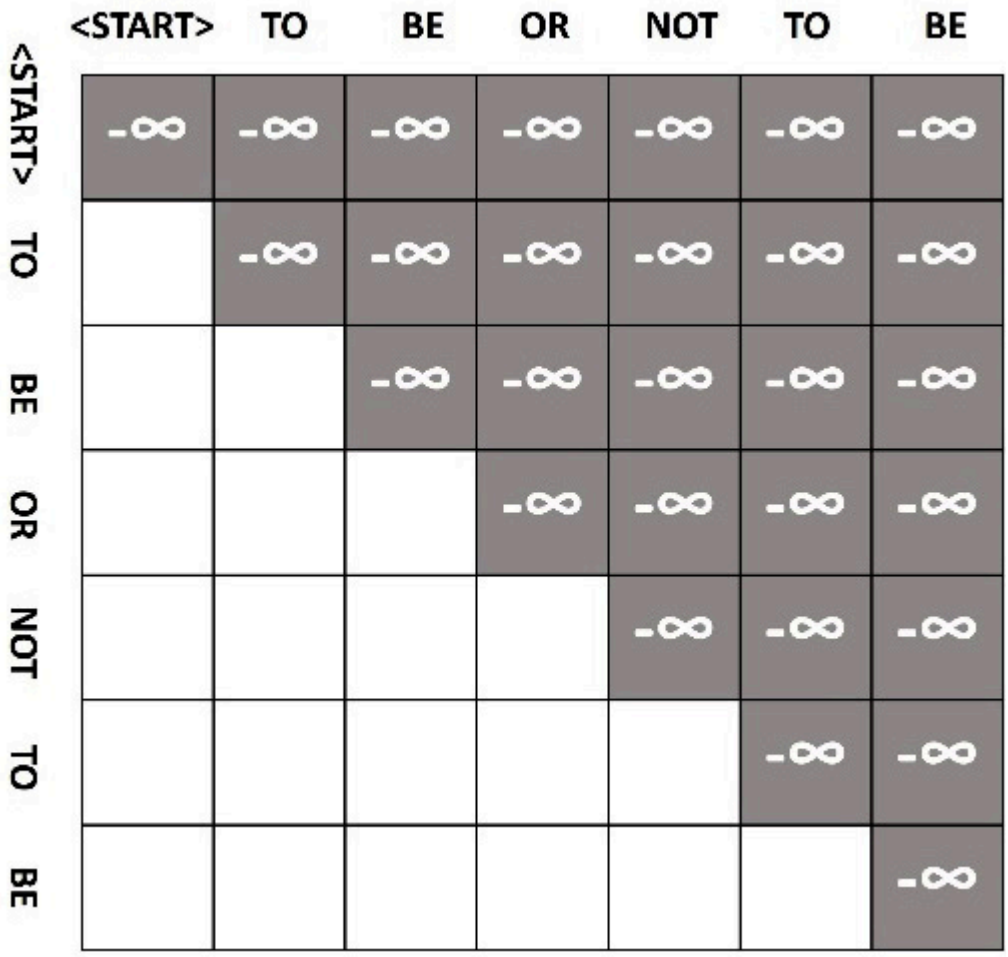



Figure 2.12 – Masked attention

The first transformer consisted of an encoder and decoder, but today there are also models that are either encoder-only or decoder-only. Today, for generative AI, they are practically all decoder-only. We have our model; now, how can you train a system that seems so complex? In the next section, we will see how to succeed at training.

# Training a transformer

How do you train such a complex model? The answer to this question is simpler than you might think. The fact that the model can learn through multi-head self-attention complex and diverse relationships allows the model to be able to be flexible and able to learn complex patterns. It would be too expensive to build examples (or find them) to teach these complex relationships to the model. So, we want a system that allows the model to learn these relationships on its own. The advantage is that if we have a large amount of text available, the model can learn without the need for us to curate the training corpus. Thanks to the advent of the in-

ternet, we have the availability of huge corpora that allow models to see text examples of different topics, languages, styles, and more.

Although the original model was a **`seq2seq`** model, later transformers (such as LLMs) were trained as language models, especially in a **self-supervised manner**. In language modeling, we consider a sequence of word *s*, and the probability of the next word in the sequence *x* is P(w|h). This probability depends on the words up to that point. By the chain rule of the probability, we can decompose this probability:

Pwh=Pwnw1:n-1=∏i=1nPwiw1:i-1

This allows us to calculate the conditional probability of a word from a sequence of previous words. The idea is that when we have enough text we can take a sequence such as **"to be or not to"** as input and have the model estimate the probability for the next word to be **"be,"** $P(be|tobeornotto)$. Then after the transformer block sequence, we have a layer that conducts a linear projection and a **softmax layer** that generates the output. The previous sequence is called context; the context length of the first transformers was 512 tokens. The model generates an output, which is a probability vector of dimension *V* (the model vocabulary), also called a **logit vector**. The projection layer is called an **unembedder** (it does reverse mapping) because we have to go from a dimension *N* tokens x *D* embedding to 1 x *V*. Since the input and output of each transformer block are the same, we could theoretically eliminate blocks and attach an unembedder and softmax to any intermediate block. This allows us to better interpret the function of each block and its internal representation.

Once we have this probability vector, we can use self-supervision for training. We take a corpus of text (unannotated) and train the model to minimize the difference between the probability of the true word in the sequence and the predicted probability. To do this, we use **cross-entropy loss** (the difference between the predicted and true probability distribution). The predicted probability distribution is the logit vector, while the true one is a one-hot encoder vector where it is 1 for the next word in the sequence and 0 elsewhere.

LCE=-∑w∈Vytwlogy^t[w]

In practice, it is simplified during training simply between the probability of the actual predicted word and 1. The process is iterative for each word in the word sequence (and is called teacher forcing). The final loss is the average over the entire sequence.

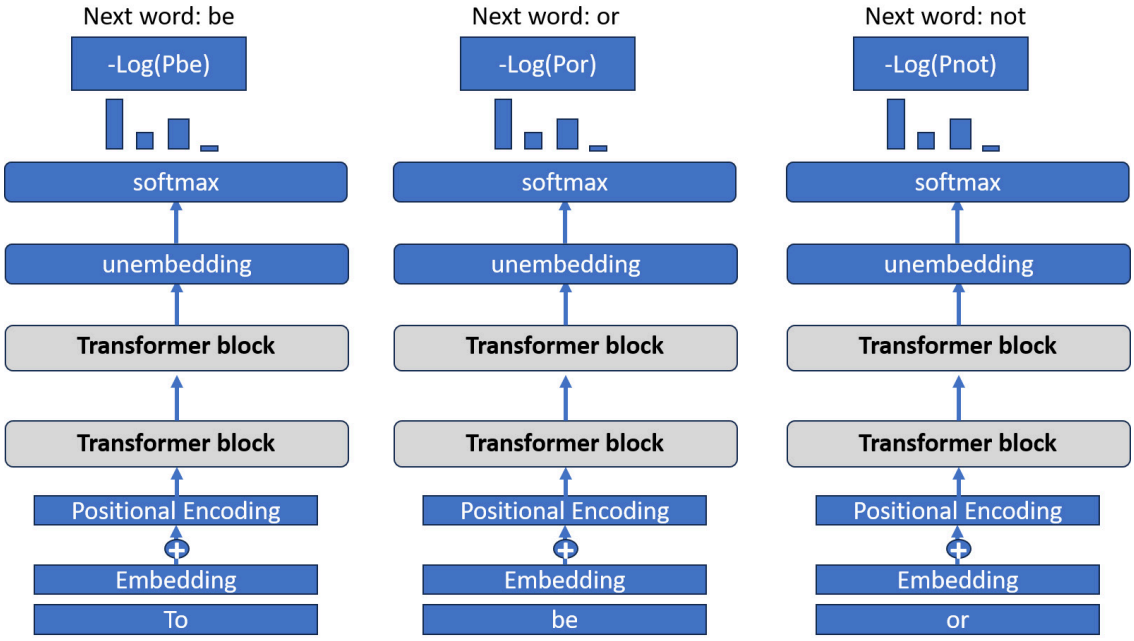



Figure 2.13 – Training of the transformer; the loss is the average of the loss of all the time steps

Since all calculations can be done in parallel in the transformer, we do not have to calculate word by word, but we fed the model with the whole sequence.

Once we have obtained a probability vector, we can choose the probability most (**greedy decoding**). Greedy decoding is formally defined as choosing the token with the highest probability at each time step:

wt=argmaxw∈VP(w|w<t)

In fact, it is no longer used because the result is too predictable, generic, and repetitive. So, more sophisticated and less deterministic sampling methods are used. This sampling process is called decoding (or autoregressive generation or causal language modeling, since it is derived from previous word choice). This system, in the simplest version, is based ei-

ther on generating the text of at most a predetermined sequence length, or as long as an end-of-sentence token (**<EOS>**) is selected.

We need to find a way to be able to select tokens while balancing both quality and diversity. A model that always chooses the same words will certainly have higher quality but will also be repetitive. There are different methods of doing the sampling:

- **Random sampling**: The model chooses the next token randomly. The sentences are strange because the model chooses rare or singular words.
- **Top-k sampling**: At each step, we sort the probabilities and choose the top *k* most likely words. We renormalize the probability and choose one at random.
- **Top-p sampling**: This is an alternative in which we keep only a percentage of the most likely words.
- **Temperature sampling**: Before **`softmax`**, we divide by a temperature parameter (between 0 and 1). The closer *t* is to 0, the closer the probability of the most likely words is to 1 (close to greedy sampling). In some cases, we can also have *t* greater than 1 when we want a less greedy approach.

So far, we have considered the fixed vocabulary and assumed that each token was a word. In general, once the model is trained, there might be some words that the model does not know to which a special token, **<UNK>**, is assigned. In transformers and LLMs afterward, a way was sought to solve the unknown word problem. For example, in the training set, we might have words such as *big*, *bigger*, and *small* but not *smaller*. *Smaller* would not be known by the model and would result in **<UNK>**. Depending on the training set, the model might have incomplete or outdated knowledge. In English, as in other languages, there are definite morphemes and grammatical rules, and we would like the tokenizer to be aware. To avoid too many **<UNK>** one solution is to think in terms of subwords (tokens).

One of the most widely used is **Byte-Pair Encoding** (**BPE**). The process starts with a list of individual characters. The algorithm then scans the

entire corpus and begins to merge the symbols that are most frequently found together. For example, we have **E** and **R**, and after the first scan, we add a new **ER** symbol to the vocabulary. The process continues iteratively to merge and create new symbols (longer and longer character strings). Typically, the algorithm stops when it has created *N* tokens (with *N* being a predetermined number at the beginning). In addition, there is a special end-of-word symbol to differentiate whether the token is inside or at the end of a word. Once the algorithm arrives at creating a vocabulary, we can segment the corpus with the tokenizer and for each subword, we assign an index corresponding to the index in the vocabulary.

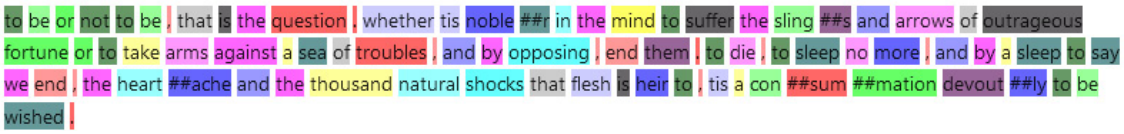



Figure 2.14 – Example of the results of tokenization

This approach generally causes common words to be present in the model vocabulary while rare words are split into subwords. In addition, the model also learns suffixes and prefixes, and considers the difference between *app* and the *app#* subword, representing a complete word and a subword (*app#* as a subword of *application*).

# Exploring masked language modeling

Although the transformer was revolutionary, the popularization of the transformer in the scientific community is also due to the **Bidirectional Encoder Representations from Transformers** (**BERT**) model. This is because BERT was a revolutionary variant of the transformer that showed the capabilities of this type of model. BERT was revolutionary because it was already prospectively designed specifically for future applications (such as question answering, summarization, and machine translation). In fact, the original transformer analyzes the left-to-right sequence, so when the model encounters an entity, it cannot relate it to what is on the right of the entity. In these applications, it is important to have context from both directions.

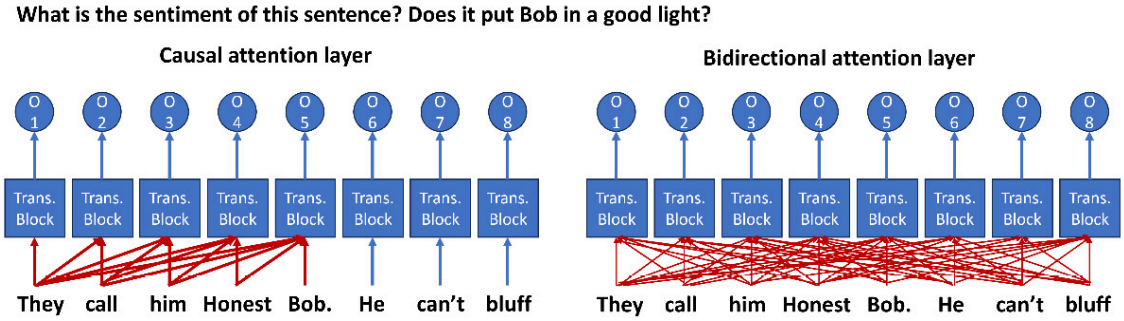



Figure 2.15 – Difference between a causal and bidirectional language model

**Bidirectional encoders** resolve this limitation by allowing the model to find relationships over the entire sequence. Obviously, we can no longer use a language model to train it (it will be too easy to identify the next word in the sequence when you already know the answer) but we have to find a way to be able to train a bidirectional model. For clarification, the model reads the entire sequence at once and, in this case, consists of the encoder only.

To try to minimize changes to the structure we use what is called the **Masked Language Model** (**MLM**) **objective**. Instead of predicting the next word in the sequence, we mask some tokens and the model has to use the rest of the sequence to predict them. Given the entire context (the model now has access to the entire sequence), BERT must predict the token that has been masked with a special token (usually called **<MASK>**). In the original training, they masked 15 % of the tokens randomly. Notice that, in this case, we do not mask the future because we want the model to be aware of the whole context. Also, to better separate the different sentences, we have a special token, **[CLS]**, that signals the beginning of an input, and **[SEP]** to separate sentences in the input (for example, if we have a question and an answer). Otherwise, the structure is the same: we have an embedder, a position encoder, different transformer blocks, a linear projection, and a softmax. The loss is calculated in the same way; instead of using the next token, we use the masked token. The original article introduced two versions of BERT: BERT-BASE (12 layers, hidden size with d=768, 12 attention heads, and 110M total parameters) and BERT-LARGE (24 layers, hidden size with d=1024, 24 attention heads, and 340M total parameters).

MLM is a flexible approach because the idea is to corrupt the input and ask the model to rebuild. We can mask, but we can also reorder or conduct other transformations. The disadvantage of this method is that only 15 percent of the tokens are actually used to learn, so the model is highly inefficient.

The training is also highly flexible. For example, the model can be extended to **next sentence prediction**, where the task is to predict whether two pairs of sentences are related (paraphrase, coherence, and entailment). In this case, BERT was trained with training pairs of sentences positively related and unrelated sentences (we exploit the `[SEP]` token between sentences). The last layer is a softmax for sentence classification; we consider the loss over the categories. This shows how the system is flexible and can be adapted to different tasks.

One final clarification. Until 2024, it was always assumed that these models were not capable of generating text. In 2024, two studies showed that by adapting the model, you can generate text even with a BERT-like model. For example, in this study, they show that one can generate text by exploiting a sequence of [MASK] tokens.

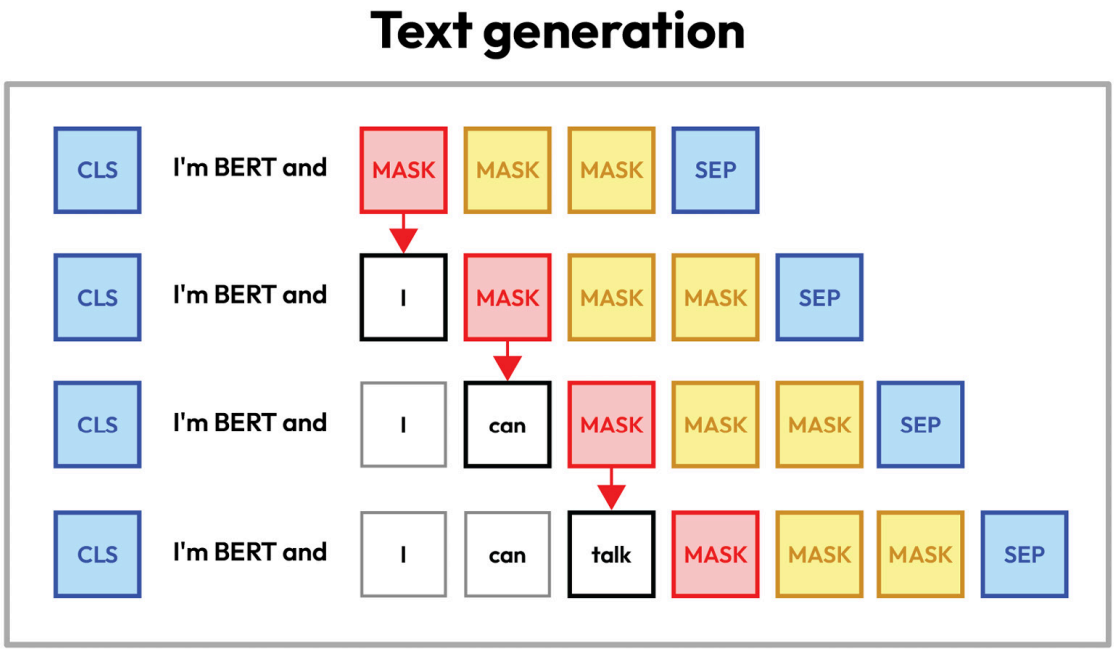



Figure 2.16 – Text generation with MLM (https://arxiv.org/pdf/2406.04823)

Now that we have seen the two main types of training for a transformer, we can better explore what happens inside these models.

# Visualizing internal mechanisms

We have seen the inner workings of the transformer, how it can be trained, and the main types of models. The beauty of attention is that we can visualize these relationships, and in this section, we will see how to do that. We can then visualize the relationships within the BERT attention head. As mentioned, in each layer, there are several attention heads and each of them learns a different representation of the input data. The color intensity indicates a greater weight in the attention weights (darker colors indicate weights that are close to 1).

We can do this using the BERTviz package:

```
head_view(attention, tokens, sentence_b_start)
```

> *IMPORTANT NOTE*
>
> *The visualization is interactive. The code is in the repository. Try running it using different phrases and exploring different relationships between different words in the phrases. The visualization allows you to explore the different layers in the model by taking advantage of the drop-down model. Hovering over the various words allows you to see the individual weights of the various heads.*

This is the corresponding visualization:

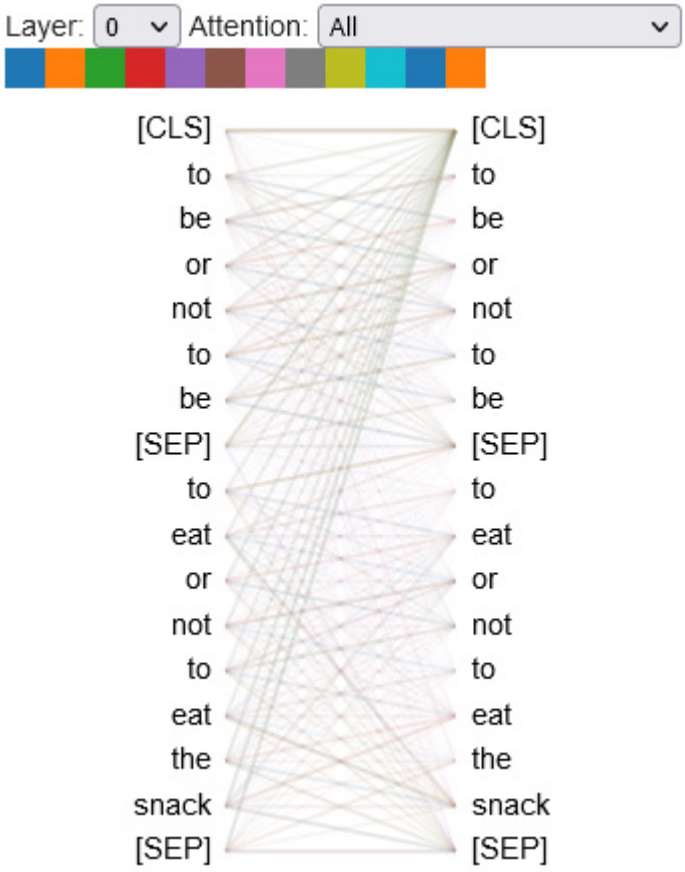



Figure 2.17 – Visualization of attention between all words in the input

We can also view the various heads of the model at the same time. This allows us to see how the various heads model different relationships. This model has 12 heads for 12 layers, so the model has 144 attention heads and can therefore see more than 100 representations for the same sentences (this explains the capacity of a model). Moreover, these representations are not completely independent; information learned from earlier layers can be used by later layers:

```
model_view(attention, tokens, sentence_b_start)
```

> *IMPORTANT NOTE*
>
> *The visualization is interactive. The code is in the repository. Try running it using different phrases and exploring different relationships. Here, we have the ensemble representation of the various attention heads. Observe how each head has a different function and how it models a different representation of the same inputs.*

This is the corresponding visualization:

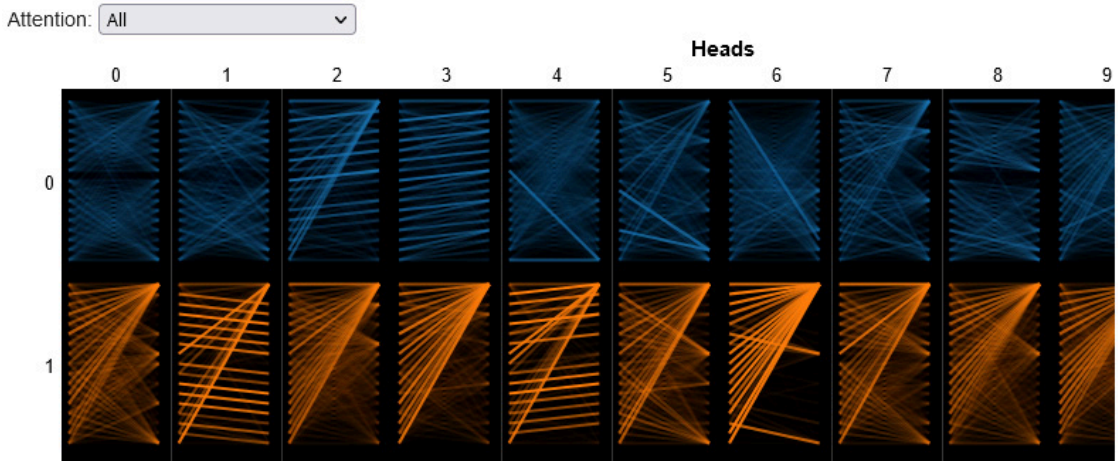



Figure 2.18 – Model view of the first two layers

Another model that has been fundamental to the current development of today's models is **Generative Pre-Trained Transformer 2 (GPT-2)**. GPT-2 is a causal (unidirectional) transformer pre-trained using language modeling on a very large corpus of ~40 GB of text data. GPT-2 was specifically trailed to predict the next token and to generate text with an input (it generates a token at a time; this token is then added to the input sequence to generate the next in an autoregressive process). In addition, this is perhaps the first model that has been trained with a massive amount of text. In addition, this model consists only of the decoder. GPT-2 is a family of models ranging from 12 layers of GPT-2 small to 48 layers of GPT-2 XL. Each layer consists of masked self-attention and a feed-forward neural network.

GPT-2 is generative and trained as a language model so we can give it an input judgment and observe the probability for the next token. For example, using "To be or not to" as input, the token with the highest probability is "be."

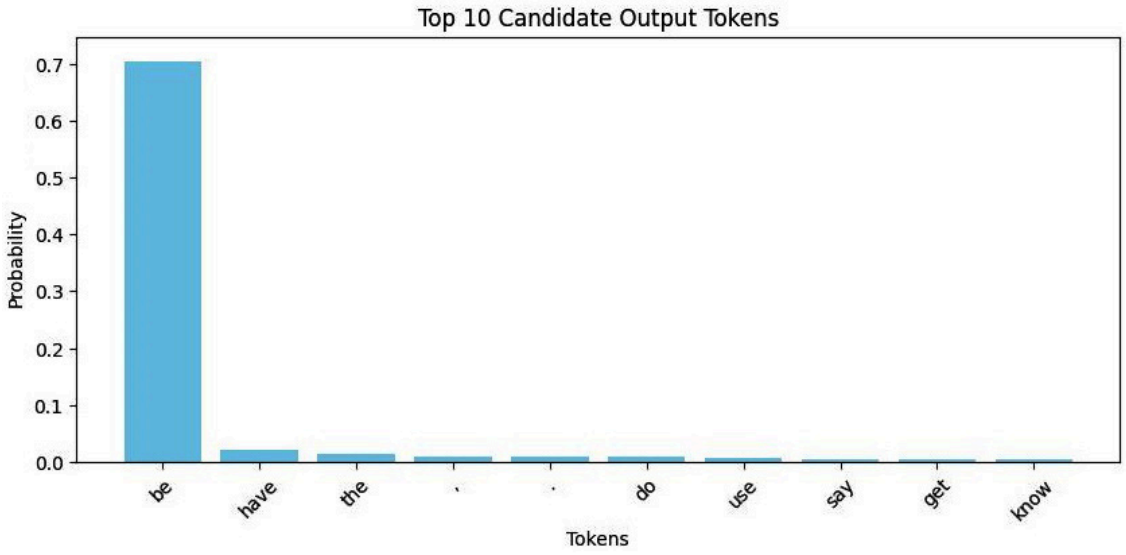



Figure 2.19 – Probabilities associated with the next token for the GPT-2 model when probed with the “To be or not to” input sequence

Sometimes, it may be necessary to understand which tokens are most important to the model to generate the next token. **Gradient X input** is a technique originally developed for convolutional networks; at a given time step, we take the output probabilities for each token, select the tokens with the highest probability, and compute the gradient with respect to the input up to the input tokens. This gives us the importance of each token to generate the next token in the sequence (the rationale is that small changes in the input tokens with the highest importance carry the largest changes in the output). In the figure, we can see the most important tokens for the next token in the sequence:

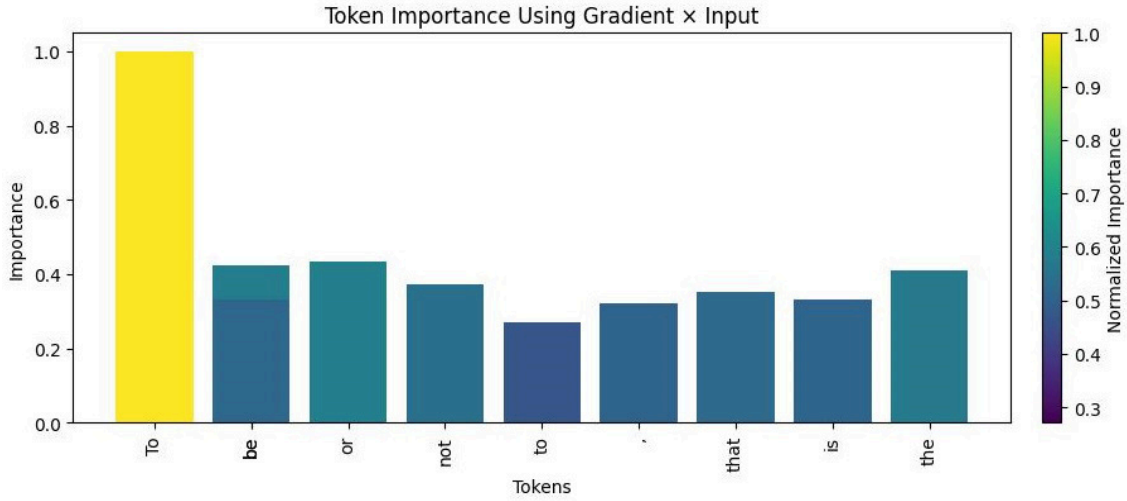



Figure 2.20 – Gradient X input for the next token in the sequence

As mentioned before, there is not only self-attention but also feedforward neural network, which plays an important role (it provides a significant

portion of the parameters in the transformer block, about 66%). Therefore, several works have focused on examining the firings of neurons in layers (this technique was also originally developed for computer vision).

We can follow this activation after each layer, and for each of the tokens, we can monitor what their rank (by probability) is after each layer. As we can see, the model understands from the first layers which token is the most likely to continue a sequence:

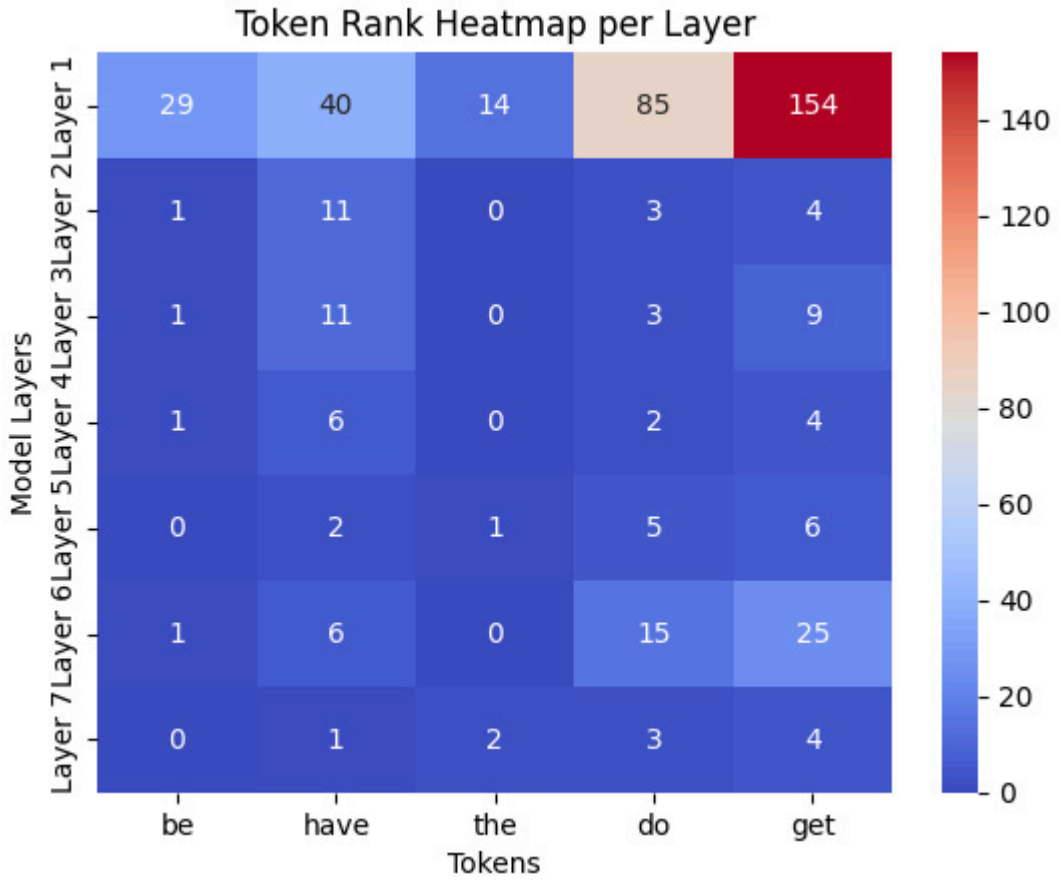



Figure 2.21 – Heatmap of the rank for the top five most likely tokens after each layer

Since there are a considerable number of neurons, it is complex to be able to observe them directly. Therefore, one way to investigate these activations is to first reduce dimensionality. To avoid negative activations, it is preferred to use **Non-Negative Matrix Factorization** (**NMF**) instead of **Principal Component Analysis** (**PCA**). The process first captures the activation of neurons in the FFNN layers of the model and is then decomposed into some factors (user-chosen parameters). Next, we can interactively observe the factors with the highest activation when a token has

been generated. What we see in the graph is the factor excitation for each of the generated tokens:

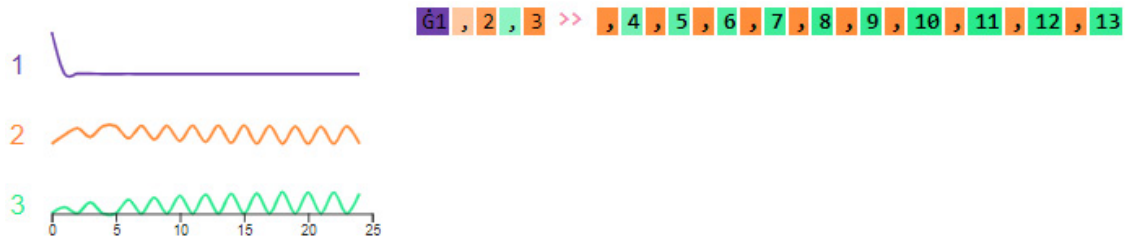



Figure 2.22 – NMF for the activations of the model in generating a sequence

We can also conduct this analysis for a single layer. This allows us to analyze interesting behaviors within the neurons of a layer (in the image layer 0 of the model). In this case, there are certain factors that focus on specific portions of the text (beginning, middle, and end). As we mentioned earlier, the model keeps track of word order in a sequence due to positional encoding, and this is reflected in activation. Other neurons, however, are activated by grammatical structures (such as conjunctions, articles, and so on). This indicates to us a specialization of what individual neurons in a pattern track and is one of the strength components of the transformer. By increasing the number of facts, we can increase the resolution and better understand what grammatical and semantic structures the pattern encodes in its activations. Moving forward in the structure of the model, we can see that layers learn a different representation.




Figure 2.23 – NMF for the activations of the model in generating a sequence

We have seen how to build a transformer and how it works. Now that we know the anatomy of a transformer, it is time to see it at work.

# Applying a transformer

The power of a transformer lies in its ability to be able to learn from an enormous amount of text. During this phase of training (called **pre-training**), the model learns general rules about the structure of a language. This general representation can then be exploited for a myriad of applications. One of the most important concepts in deep learning is **transfer learning**, in which we exploit the ability of a model trained on a large amount of data for a task different from the one it was originally trained for. A special case of transfer learning is **fine-tuning**. Fine-tuning allows us to adapt the general knowledge of a model to a particular case. One way to do this is to add a set of parameters to a model (at the top of it) and then train these parameters by gradient descent for a specific task.

The transformer has been trained with large amounts of text and has learned semantic rules that are useful in understanding a text. We want to exploit this knowledge for a specific application such as sentiment classification. Instead of training a model from scratch, we can adapt a pre-trained transformer to classify our sentences. In this case, we do not want to destroy the internal representation of the model but preserve it. That is why, during fine-tuning, most of the layers are frozen (there is no update on the weights). Instead, we just train those one or two layers that we add to the top of the model. The idea is to preserve the representation and then learn how to use it for our specific task. Those two added layers learn precisely how to use the internal representation of the model. To give a simple example, let's imagine we want to learn how to write scientific papers. To do that, we don't have to learn how to write in English again, just to adapt our knowledge to this new task.

In BERT, as we mentioned, we add a particular token to the beginning of each sequence: a `[CLS]` token. During training or even inference in a bidirectional transformer, this token waits for all others in the sequence (if you remember, all tokens are connected). This means that the final vector (the one in the last layer) is contextualized for each element in the sequence. We can then exploit this vector for a classification task. If we

have three classes (for example, positive, neutral, and negative) we can take the vector for a sequence and use softmax to classify.

y=softmaxWhCLS

The model was not originally trained for sequence classification, so we'd like to introduce a learnable matrix to enable class separation. This matrix represents a linear transformation and can alternatively be implemented using one or more linear layers. We then apply a cross-entropy loss function to optimize these weights. This setup follows the standard supervised learning paradigm, where labeled data is used to adapt the transformer to a specific task.

In this process, we have so far assumed that the remainder of the transformer's weights remain frozen. However, as observed in convolutional neural networks, even minimal fine-tuning of model parameters can enhance performance. Such updates are typically carried out with a very low learning rate.

We can adapt a pre-trained transformer for new tasks through supervised fine-tuning.

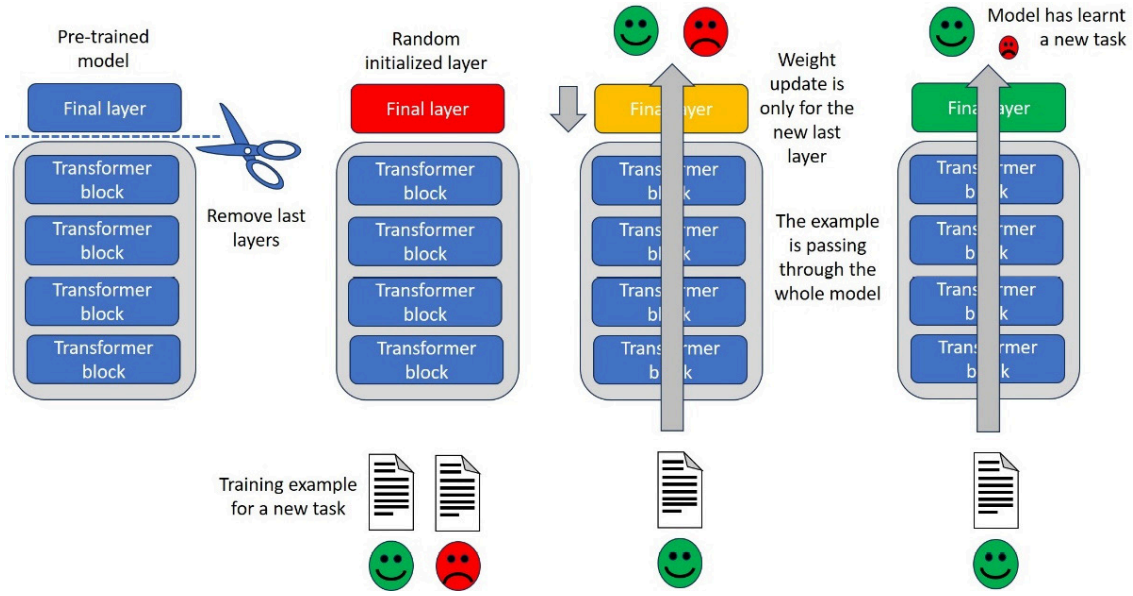



Figure 2.24 – Fine-tuning a transformer

In the first step, we are removing the final layer (this is specific to the original task). In the second step, we add a random initialized layer and gather training examples for the new task. During the fine-tuning, we are

presenting the model with new examples (in this case, positive and negative reviews). While keeping the model frozen (each example is processed by the whole model in the forward pass), we update the weight only in the new layer (through backpropagation). The model has now learned the new task.

Conducting finetuning with Hugging Face is a straightforward process. We can use a model such as distill-BERT (a distilled version of BERT) with a few lines of code and the dataset we used in the previous chapter. We need to prepare the dataset and tokenize it (so that it can be used with a transformer). Hugging Face then allows with a simple wrapper that we can train the model. The arguments for training are stored in **`TrainingArguments`**:

```
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    evaluation_strategy="epoch"
)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset
)
trainer.train()
```

> *IMPORTANT NOTE*

> *Notice that the process is very similar to training a neural network. In fact, the transformer is a deep learning model; for the training, we are using similar hyperparameters.*

In this case, we used only a small fraction of the reviews. The beauty of fine-tuning is that we need only a few examples to have a similar (if not better) performance than a model trained from scratch.

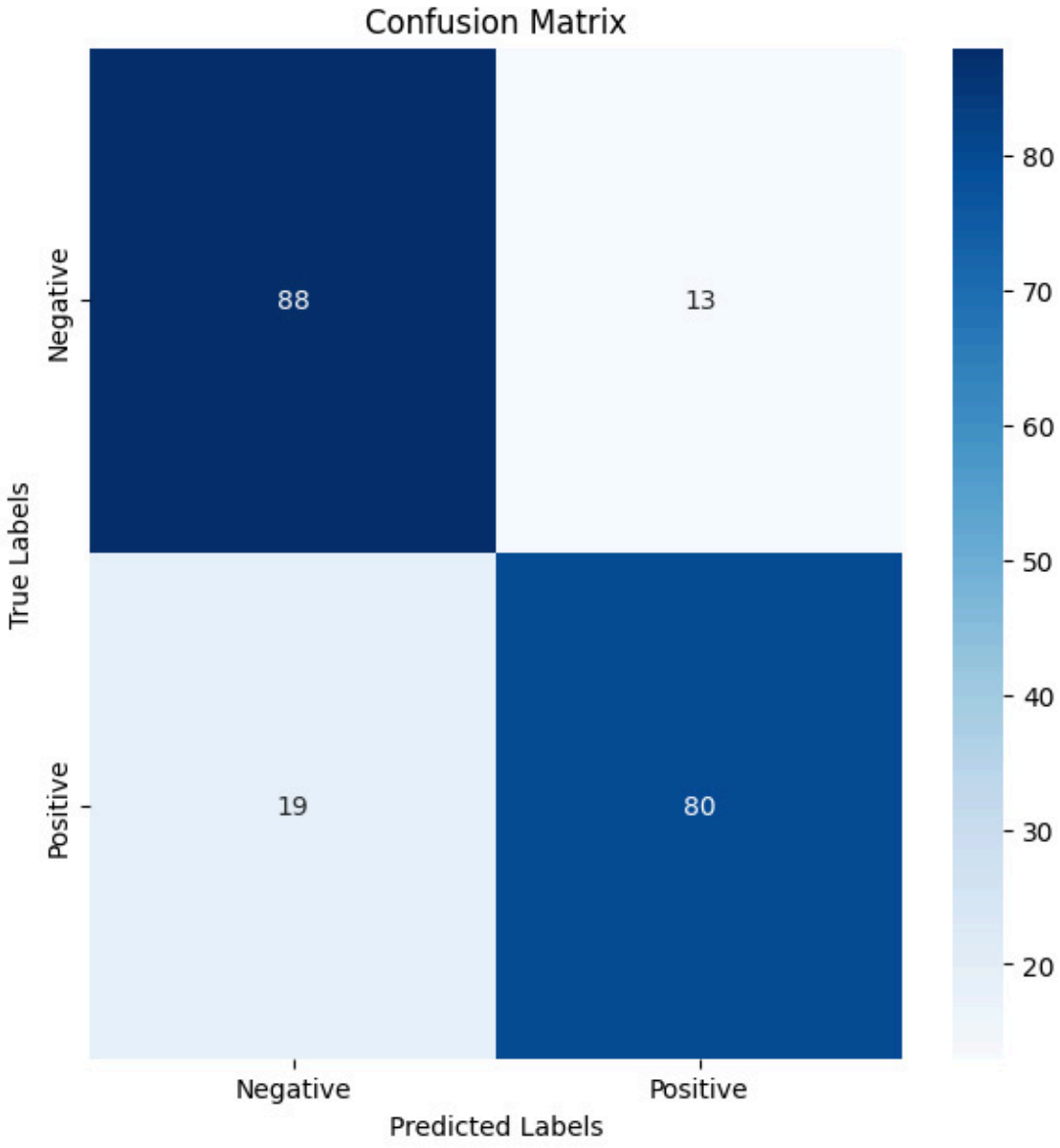



Figure 2.25 – Confusion matrix after fine-tuning

BERT's training was done on 64 TPUs (special hardware for tensor operations) for four days; this is beyond the reach of most users. In contrast, fine-tuning is possible either on a single GPU or on a CPU. As a result, BERT achieved state-of-the-art performance upon its release across a wide array of tasks, including paraphrase detection, question answering, and sentiment analysis. Hence, several variants such as **RoBERTa** and **SpanBERT** (in this case, we mask an entire span instead of a single token with better results) or adapted to specific domains such as **SciBERT** were

born. However, encoders are not optimal for generative tasks (because of mask training) while decoders are.

To conduct machine translation, the original transformer consisted of an encoder and a decoder. A model such as GPT-2 only has a decoder. We can conduct fine-tuning in the same way as seen before, we just need to construct the dataset in an optimal way for a model that is constituted by the decoder alone. For example, we can take a dataset in which we have English and French sentences and build a dataset for finetuning as follows: **`<sentence in English>`** followed by a special **`<to-fr>`** token and then the **`<sentence in French>`**. The same approach can be used to teach summarization to a model, where we insert a special token meaning summarization. The model is fine-tuned by conducting the next token prediction (language modeling).

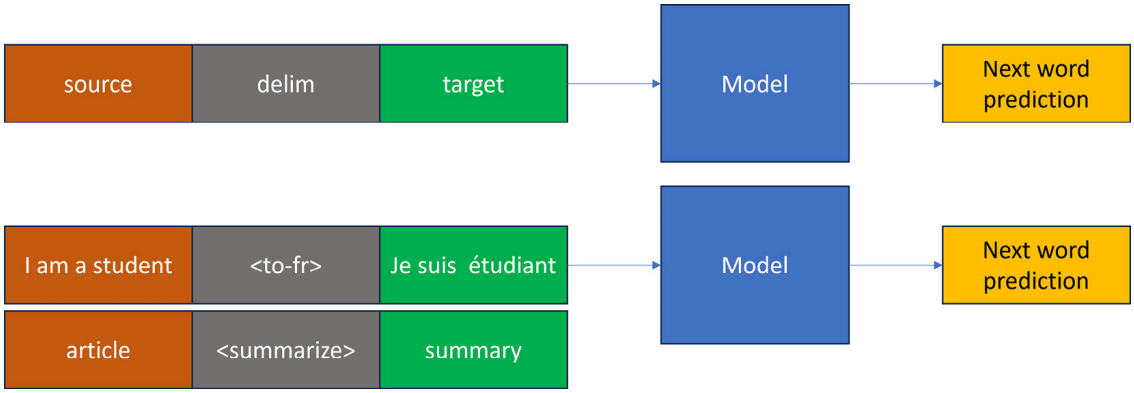



Figure 2.26 – Fine-tuning of a decoder-only model

Another way to exploit the learned knowledge of a model is to use **knowledge distillation**. In the previous section, we used distillGPT-2 which is a distilled version of GPT-2. A distilled model captures knowledge from a much larger model without losing significant performance but is much more manageable. Models that are trained with a large amount of text learn a huge body of knowledge. All this knowledge and skill is often redundant when we need a model for some specific task. We are interested in having a model that is very capable for a task, but without wanting to deal with a model of billions of parameters. In addition, sometimes we do not have enough examples for a model to learn the task from scratch. In this case, we can extract knowledge from the larger model.

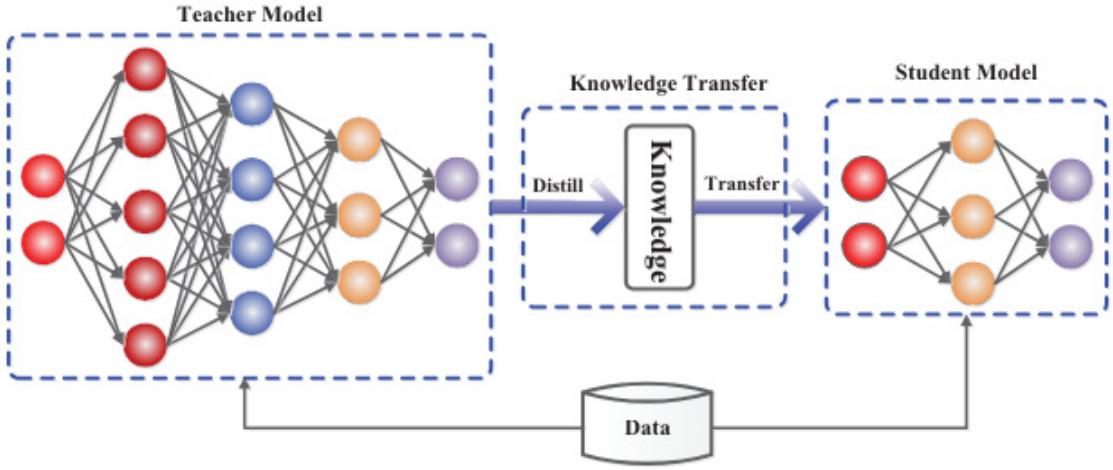



Figure 2.27 – Generic teacher-student framework for knowledge distillation (https://arxiv.org/pdf/2006.05525)

Knowledge distillation can be seen as a form of compression, in which we try to transfer knowledge from a trained "teacher" model with many parameters to a "student" model with fewer parameters. The student model tries to mimic the teacher model and achieve the same performances as the teacher model in a task. In such a framework, we have three components: the models, knowledge, and algorithm. The algorithm can exploit either the teacher's logits or intermediate activations. In the case of the logits, the student tries to mimic the predictions of the teacher model, so we try to minimize the difference between the logits produced by the teacher and the student. To do this, we use a distillation loss that allows us to train the student model.

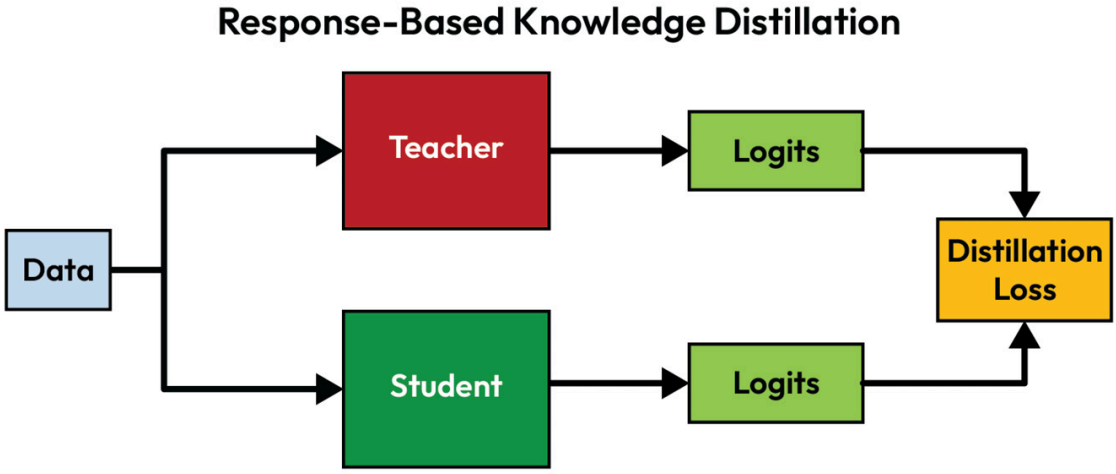



Figure 2.28 – Teacher-student framework for knowledge distillation training (https://arxiv.org/pdf/2006.05525)

For knowledge distillation, the steps are also similar. The first step is data preprocessing. For each model, you must remember to choose the model-

specific tokenizer (although the one from GPT-2 is the most widely used many models have different tokenizers). We must then conduct fine-tuning of a model on our task (there is no model that is specific to classify reviews). This model will be our teacher. The next step is to train a student model. We can also use a pre-trained model that is smaller than the teacher (this allows us to be able to use a few examples to train it).

One important difference is that we now have a specific loss for knowledge distillation. This distillation loss calculates the loss between the teacher's logits and the student's logits. This function typically uses the Kullback-Leibler divergence loss to calculate the difference between the two probability distributions (Kullback-Leibler divergence is really a measure of the difference between two probability distributions). We can define it as follows:

```
def distillation_loss(outputs_student, outputs_teacher,
                      temperature=2.0):
    log_prob_student = F.log_softmax(
        outputs_student / temperature, dim=-1)
    prob_teacher = F.softmax(
        outputs_teacher / temperature, dim=-1)
    loss = KLDivLoss(reduction='batchmean')(
        log_prob_student, prob_teacher)
    return loss
```

At this point, we just have to have a way to train our system. In this case, the teacher will be used only in inference while the student model will be trained. We will use the teacher's logits to calculate the loss:

```
def train_epoch(model, dataloader, optimizer, device,
                teacher_model, temperature=2.0):
    model.train()
    total_loss = 0
    for batch in tqdm(dataloader, desc="Training"):
        inputs = {k: v.to(device)
                  for k, v in batch.items()
                  if k in ['input_ids', 'attention_mask']}
        with torch.no_grad():
            outputs_teacher = teacher_model(**inputs).logits
```

```
            outputs_student = model(**inputs).logits
            loss = distillation_loss(
                outputs_student, outputs_teacher, temperature)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        return total_loss / len(dataloader)
```

As can be seen in the following figure, the performance of the student model is similar to the teacher model:

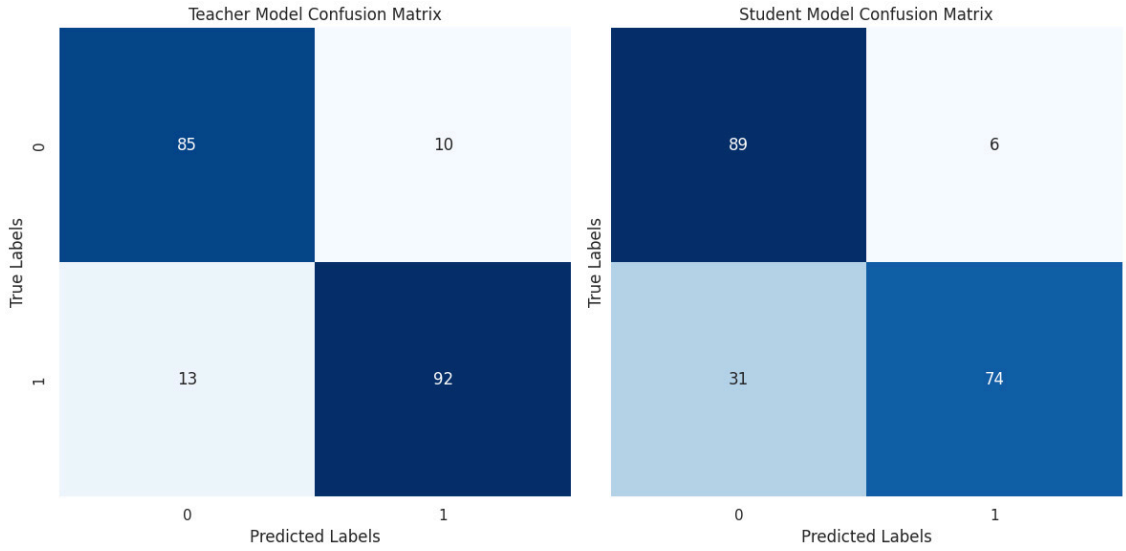



Figure 2.29 – Confusion matrix for the teacher and student model

Fine-tuning and knowledge distillation allow us to be able to use a transformer for any supervised task. Fine-tuning allows us to work with datasets that are small (and where there are often too few examples to train a model from scratch). Knowledge distillation, on the other hand, allows us to get a smaller model (but performs as well as a much larger one) when the computational cost is the limit. By taking advantage of these techniques, we can tackle any task.

# Summary

In this chapter, we discussed the transformer, the model that revolutionized NLP and artificial intelligence. Today, all models that have commercial applications are derivatives of the transformer, as we learned in this

chapter. Understanding how it works on a mechanistic level, and how the various parts (self-attention, embedding, tokenization, and so on) work together, allows us to understand the limitations of modern models. We saw how it works internally in a visual way, thus exploring the motive of modern artificial intelligence from multiple perspectives. Finally, we saw how we can adapt a transformer to our needs using techniques that leverage prior knowledge of the model. Now we can repurpose this process with virtually any dataset and any task.

Learning how to train a transformer will allow us to understand what happens when we take this process to scale. An LLM is a transformer with more parameters and that has been trained with more text. This leads to emergent properties that have made it so successful, but both its merits and shortcomings lie in the elements we have seen.

In ***Chapter 3***, we will see precisely how to obtain an LLM from a transformer. What we have learned in this chapter will allow us to see how this step comes naturally.

Table of contents collapsed